

```
In [ ]: import json
import marimo as mo
import numpy as np
import pandas as pd
import shap
import xgboost as xgb
from pathlib import Path
from sklearn.calibration import CalibratedClassifierCV

from klarna_ds_case_study.training.config import CATEGORICAL_FEATURES
from klarna_ds_case_study.training.pipeline import (
    load_training_data,
    split_data,
    compute_metrics,
)
```

SHAP Analysis & Monotone Constraints

Goals:

1. Inspect SHAP values on the minimal feature set to verify features are used as expected
2. Identify features with counter-intuitive directions
3. Enforce monotone constraints where needed and compare performance

1. Load data and train model on minimal feature set

```
In [ ]: FEATURES_TO_REMOVE = [
    "num_failed_payments_6m",
    "amount_repaid_1m",
    "num_confirmed_payments_3m",
    "amount_repaid_3m",
    "amount_repaid_6m",
    "new_exposure_7d",
    "amount_repaid_14d",
    "card_expiry_month",
    "new_exposure_14d",
    "num_failed_payments_3m",
]

X, y = load_training_data()
X = X.drop(columns=FEATURES_TO_REMOVE)
X_train, X_test, y_train, y_test = split_data(X, y)
```

```
In [ ]: PROJECT_ROOT = Path(__file__).resolve().parents[1]
best_params = json.loads(
    (PROJECT_ROOT / "data" / "models" / "best_xgb_params.json").read_text()
)
```

```
spw = float(np.sum(y_train == 0) / np.sum(y_train == 1))
model = xgb.XGBClassifier(**{**best_params, "scale_pos_weight": spw})
model.fit(X_train, y_train)
```

```
In [ ]: calibrated_model = CalibratedClassifierCV(model, method="isotonic", cv=5)
calibrated_model.fit(X_train, y_train)

baseline_probs = calibrated_model.predict_proba(X_test)[: , 1]
baseline_metrics = compute_metrics(y_test, baseline_probs)

mo.md(f"Baseline metrics (minimal feature set): {baseline_metrics}")
```

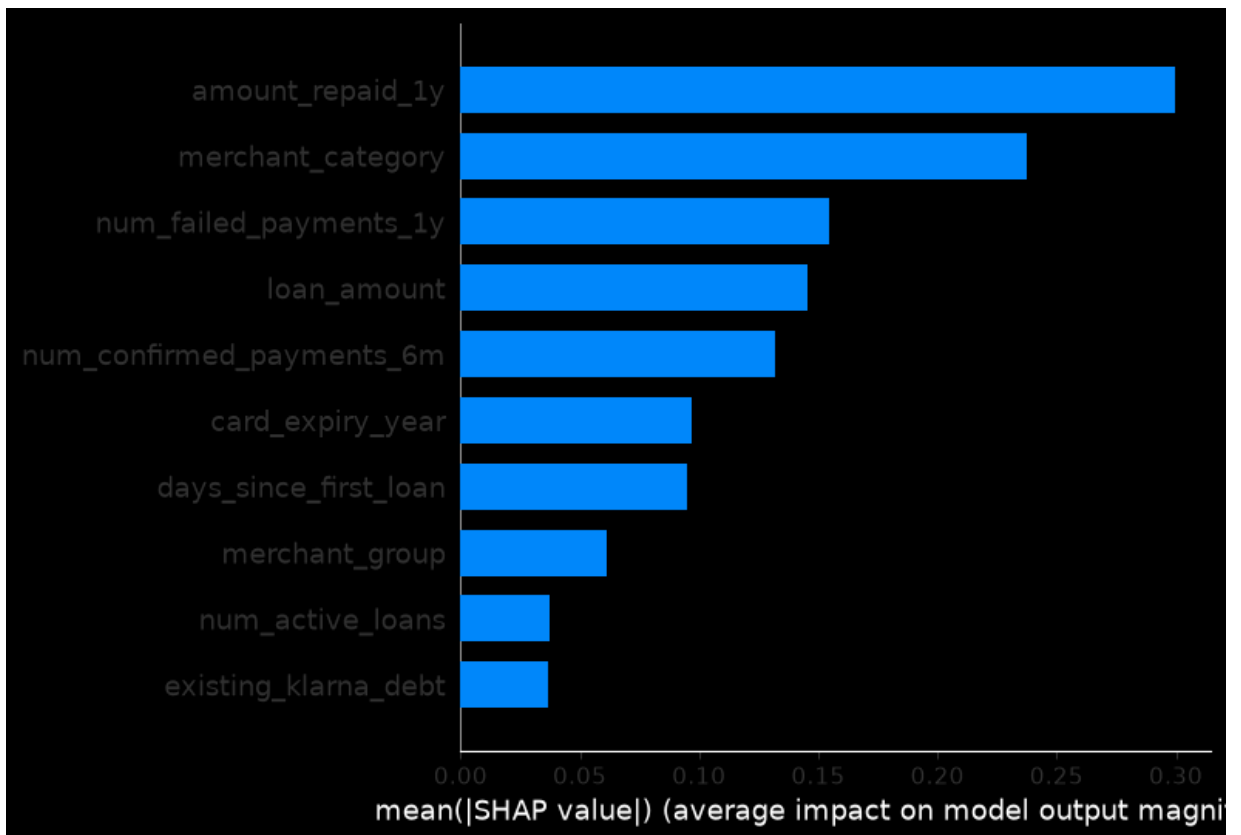
Baseline metrics (minimal feature set): {'roc_auc': 0.6896, 'pr_auc': 0.1172, 'log_loss': 0.1992, 'brier_score': 0.05, 'ece': np.float64(0.0006)}

2. SHAP values

```
In [ ]: explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_train)
```

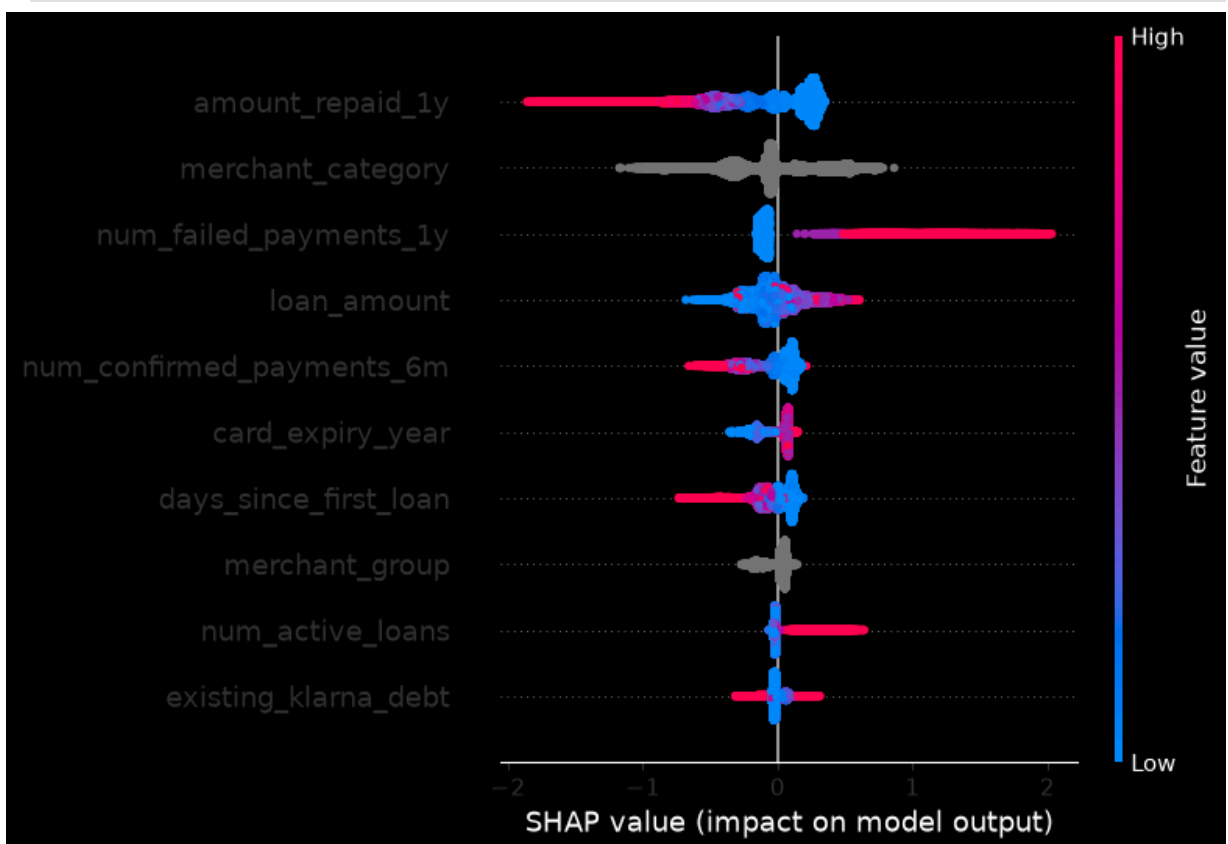
Summary plot (bar) — global feature importance

```
In [ ]: shap.summary_plot(shap_values, X_train, plot_type="bar")
```



Summary plot (beeswarm) — feature value vs SHAP direction

```
In [ ]: shap.summary_plot(shap_values, X_train)
```

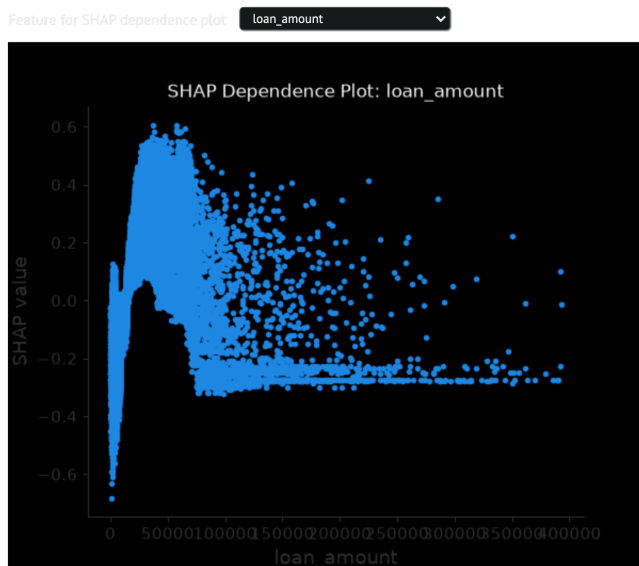


Dependence plots for key features

These show how the SHAP value of a feature varies with the feature's actual value.

```
In [ ]: shap_dependence_feature_dropdown = mo.ui.dropdown(  
    options=list(X_train.columns),  
    value=list(X_train.columns)[0],  
    label="Feature for SHAP dependence plot",  
)
```

```
In [ ]: import matplotlib.pyplot as plt  
  
_selected_feature = shap_dependence_feature_dropdown.value  
  
plt.figure()  
shap.dependence_plot(  
    _selected_feature, shap_values, X_train, show=False, interaction_index=1  
)  
_ax = plt.gca()  
_ax.set_title(f"SHAP Dependence Plot: {_selected_feature}")  
_ax.set_xlabel(_selected_feature)  
_ax.set_ylabel("SHAP value")  
  
mo.vstack(  
    [  
        shap_dependence_feature_dropdown,  
        mo.ui.matplotlib(_ax),  
    ]  
)
```



3. Expected monotonic directions

Based on (limited) domain knowledge, we define the expected direction for each numeric feature:

- **+1**: higher value → higher default risk (positive SHAP)
- **-1**: higher value → lower default risk (negative SHAP)
- **0**: no constraint (categorical or ambiguous)

We can already see that not all features are used monotonically, but we will enforce the expected directions for the next part of the analysis.

```
In [ ]: # Expected monotone directions for the minimal feature set
# +1 = more → more default risk, -1 = more → less default risk, 0 = no constraint
monotone_directions = {
    "loan_amount": 1,
    "num_active_loans": 1,
    "existing_klarna_debt": 1,
    "num_failed_payments_1y": 1,
    "num_confirmed_payments_6m": -1,
    "days_since_first_loan": -1,
    "amount_repaid_1y": -1,
    "card_expiry_year": 0,
    "merchant_group": 0,
    "merchant_category": 0,
}

monotone_constraints_tuple = tuple(
    monotone_directions.get(col, 0) for col in X_train.columns
)

mo.md(f"""
Features and their constraints:

{chr(10).join(f"- {col}: {monotone_directions.get(col, 0)}" for col in X_train.columns)
""")
```

Features and their constraints:

- loan_amount : 1
- card_expiry_year : 0
- existing_klarna_debt : 1
- num_active_loans : 1
- days_since_first_loan : -1
- num_confirmed_payments_6m : -1
- num_failed_payments_1y : 1
- amount_repaid_1y : -1
- merchant_group : 0
- merchant_category : 0

4. Retrain with monotone constraints

```
In [ ]: constrained_params = {
    **best_params,
```

```

        "scale_pos_weight": spw,
        "monotone_constraints": monotone_constraints_tuple,
    }

    model_constrained = xgb.XGBClassifier(**constrained_params)
    model_constrained.fit(X_train, y_train)

```

```

In [ ]: calibrated_constrained = CalibratedClassifierCV(
        model_constrained, method="isotonic", cv=5
    )
    calibrated_constrained.fit(X_train, y_train)

    constrained_probs = calibrated_constrained.predict_proba(X_test)[: , 1]
    constrained_metrics = compute_metrics(y_test, constrained_probs)

    comparison_df = pd.DataFrame(
        [
            {"model": "baseline (no constraints)", **baseline_metrics},
            {"model": "monotone constrained", **constrained_metrics},
        ]
    )

    mo.md(f"""
    **Performance comparison:**

    {mo.as_html(comparison_df)}
    """)

```

Performance comparison:

5. SHAP values with monotone constraints

```
In [ ]: explainer_constrained = shap.TreeExplainer(model_constrained)
shap_values_constrained = explainer_constrained.shap_values(X_train)
```

```
In [ ]: shap.summary_plot(shap_values_constrained, X_train, plot_type="bar", show=False)
```

```
In [ ]: shap.summary_plot(shap_values_constrained, X_train, show=False)
```

```
In [ ]: def _():
    import matplotlib.pyplot as plt

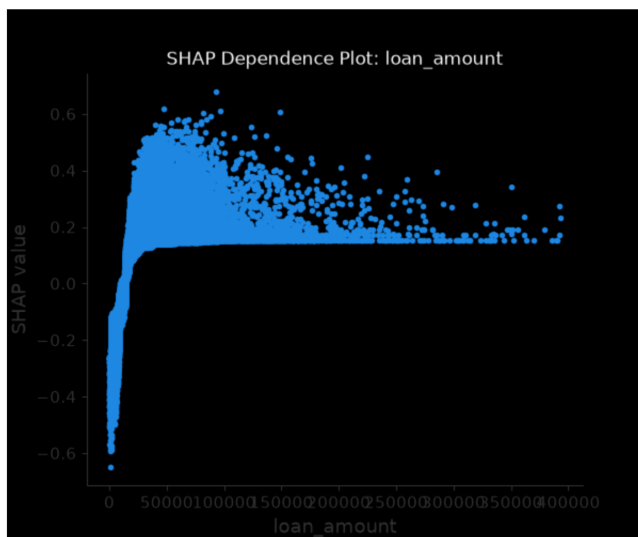
    _selected_feature = shap_dependence_feature_dropdown.value

    plt.figure()
    shap.dependence_plot(
        _selected_feature,
        shap_values_constrained,
        X_train,
        show=False,
        interaction_index=None,
    )
    _ax = plt.gca()
    _ax.set_title(f"SHAP Dependence Plot: {_selected_feature}")
    _ax.set_xlabel(_selected_feature)
    _ax.set_ylabel("SHAP value")
    return mo.vstack(
        [
            shap_dependence_feature_dropdown,
            mo.ui.matplotlib(_ax),
        ]
    )

_()
```

Feature for SHAP dependence plot

loan_amount



6. Conclusion

The performance is once again very similar once we constrain our features.

If interpretability, alignment with domain knowledge and regulatory compliance are important, enforcing monotone constraints can be a valuable tool.